



**TOBB University of Economics and Technology Department of
Computer Engineering**

**BIL 481 - Software Engineering Fall
2025 Semester**

Kalite Güvencesi Planı

NOTLA

University Lecture Notes Sharing and Evaluation Platform

Abdullah Arda Gündoğdu	231401029
Ömer Talha Akbulut	221401005
Yunus Emre Özçelik	221401001
İsmail Emre Yıldız	221401006

Delivery Date: 19.11.2025

1. Amaç ve Kapsam

Bu QA (Quality Assurance – Kalite Güvencesi) planının amacı, Notla platformunun (Django backend + React frontend) hatasız, güvenilir ve kullanılabilir şekilde çalışmasını sağlamak için izlenecek test ve kalite süreçlerini tanımlamaktır.

Sistem şu özellikleri içerir:

- Öğrencilerin ders listelerini görüntülemesi
- Dersler için not (PDF, doküman, sunum vb.) yüklemesi
- Dersler için yorum/değerlendirme ekleyebilmesi
- Kullanıcı hesap işlemleri (kayıt, giriş, profil)

Bu plan, özellikle:

- Django REST API (backend)
- Dosya yükleme (media)
- Ders ve not listeleri üzerine odaklanır.

2. Quality Assurance Stratejisi

2.1 Genel Yaklaşım

Notla için kalite güvencesi üç katmanlıdır;

1. Geliştirici Seviyesi Kontrolü:

- Kod yazarken temel kontroller (lint, tipik hatalar)
- Basit birim testleri
- Lokal ortamda API uçlarının elle test edilmesi (Postman, browser)

2. Otomatik Testler:

- Django test framework veya pytestile:
 - Model testleri (Course, Note, Review, User)
 - Serializer testleri
 - API endpoint testleri (ör: /api/courses/, /api/courses/{id}/notes/)

3. Manuel Test ve Kullanıcı Testleri:

- Ekip tarafından Postman veya frontend arayüzü ile senaryo testleri
- Küçük bir öğrenci grubu ile kullanılabilirlik testleri (2–10 kişi)
- Gerekirse basit yük testi (aynı anda çok kullanıcı not yüklerse ne olur?)

2.2 Test Ortamı

- Geliştirme Ortamı:
 - Lokal makinede: http://127.0.0.1:8000/
 - Veritabanı: PostgreSQL test/veritabanı (notla_db)
 - DEBUG: True
- Test Verisi:
 - Örnek kullanıcılar: test_student1, test_student2
 - Örnek dersler: BİL481, MAT101
 - Farklı boyutlarda ve türlerde örnek pdf/dosya:
 - Küçük pdf (1–2 sayfa)
 - Orta boyutlu pdf (5–10 MB)
 - Geçersiz dosya (bozuk pdf, desteklenmeyen uzantı vb.)

3. Test Metodolojileri

3.1 Birim Testleri (Unit Testing)

Amaç: Tek bir fonksiyonun veya sınıfın tek başına doğru çalıştığını kontrol etmek. Uygulanacağı yerler:

- Modeller:
 - Course: ders kodu ve isim zorunluluğu, created_at, updated_at alanları
 - Note: dosya türü ve boyut kontrolü, course ve user ilişkisi
 - Review: rating değerinin 1–5 arasında olması, zorunlu alanlar
- Serializer'lar:
 - CourseSerializer, CourseDetailSerializer, NoteSerializer, ReviewSerializer
 - Zorunlu alanların validasyonu (ör: rating zorunlu mu, file_type doğru mu)

Araçlar:

- pytestveya python manage.py test

3.2 Entegrasyon / API Testleri

Amaç: Endpoint'lerin uçtan uca (request–response) düzgün çalıştığını görmek. Test edilecek ana endpoint'ler:

1. Ders Listesi

- GET /api/courses/
 - Filtre ve arama parametreleri (search, ordering)
 - Boş liste durumu
- GET /api/courses/{id}/

- Geçerli id → ders detayı
- Geçersiz id → 404

2. Not Listesi ve Yükleme

- GET /api/courses/notes/veya /api/notes/
 - Filtre: courseve file_type
- POST /api/courses/{id}/notes/
 - Başarılı not yükleme (geçerli kullanıcı, geçerli dosya)
 - Dosya olmadan istek → hata mesajı
 - Desteklenmeyen dosya türü → hata mesajı (varsa)

3. Kullanıcı İşlemleri

- POST /api/auth/register/
- POST /api/auth/login/
- GET /api/auth/profile/(giriş yapmadan istek → 401 / 403)

Araçlar:

- Django Rest Framework testleri
- Postman ile manuel API istekleri

3.3 Manuel Testler

Manuel testler özellikle:

- Dosya yükleme (farklı dosya tipleri ve boyutları)
- Hata mesajlarını görmek (örneğin dosya zorunlu ama gönderilmemişse)

- Kullanıcı arayüzü (React tarafı geldiğinde) için kullanılır.

Örnek manuel senaryolar:

1. Başarılı Not Yükleme Senaryosu

- Kullanıcı giriş yapar.
- Bir derse gider.
- title, file, file_type alanlarını doldurur.
- Yükleme sonrası API'den veya arayüzden yüklenen notu görür.

2. Hatalı Dosya Senaryosu

- Kullanıcı giriş yapar.
- Derse gider, dosya seçmeden formu gönderir.
- Backend'den anlamlı bir hata mesajı beklenir (ör: "Dosya yüklenmedi!").

3. Ders Filtreleme Senaryosu

- /api/courses/?search=BIL çağrılır.
- Sadece BIL ile başlayan veya içinde BIL geçen dersler dönmelidir.

4. Otomatik vs. Manuel Test Ayrımı

Aşağıda hangi bölümün otomatik, hangisinin manuel test edileceği özetlenmiştir:

Özellik	Otomatik Test	Manuel Test
Kullanıcı kayıt / login	API testleri	Form doldurma senaryosu

Ders listesi (/api/courses/)	API + unit test	Sadece arayüz varsa
Ders detayı	API test	Arayüzden göz kontrolü
Not listesi (/api/notes/)	API filtresi testleri	Gerçek kullanıcının görmesi
Not yükleme	Geçerli/Geçersiz istekler	Farklı boyutta pdf ile deneme
Yorum / değerlendirme	API test	Örnek yorum girme, sonuç görüntüleme
Yetkilendirme (login zorunlu)	Unauthorized istek testleri	“login olmadan tıklayınca ne oluyor?”
Performans (yük testi)	Basit script/araç	Gözlem + log inceleme

5. Sonuç

Bu QA planı ile hedefimiz:

- Kritik fonksiyonların (ders listeleme, not yükleme, değerlendirme, kullanıcı girişi) otomatik testlerle sürekli olarak kontrol edilmesi,
- Gerçek kullanıcılarla küçük çaplı kullanılabilirlik testleri yapılması,
- Dosya yükleme ve ders sorgulama gibi yoğun kullanılan alanlarda hem fonksiyonel hem de basit performans testlerinin yapılmasıdır.

Bu sayede:

- “Tüm öğrencilerle tek tek test yapmadan”

- Küçük bir grup ve test verisiyle
- Gerçeğe yakın bir yük simülasyonu oluşturup sistemin kalitesini güvence altına alabiliriz.